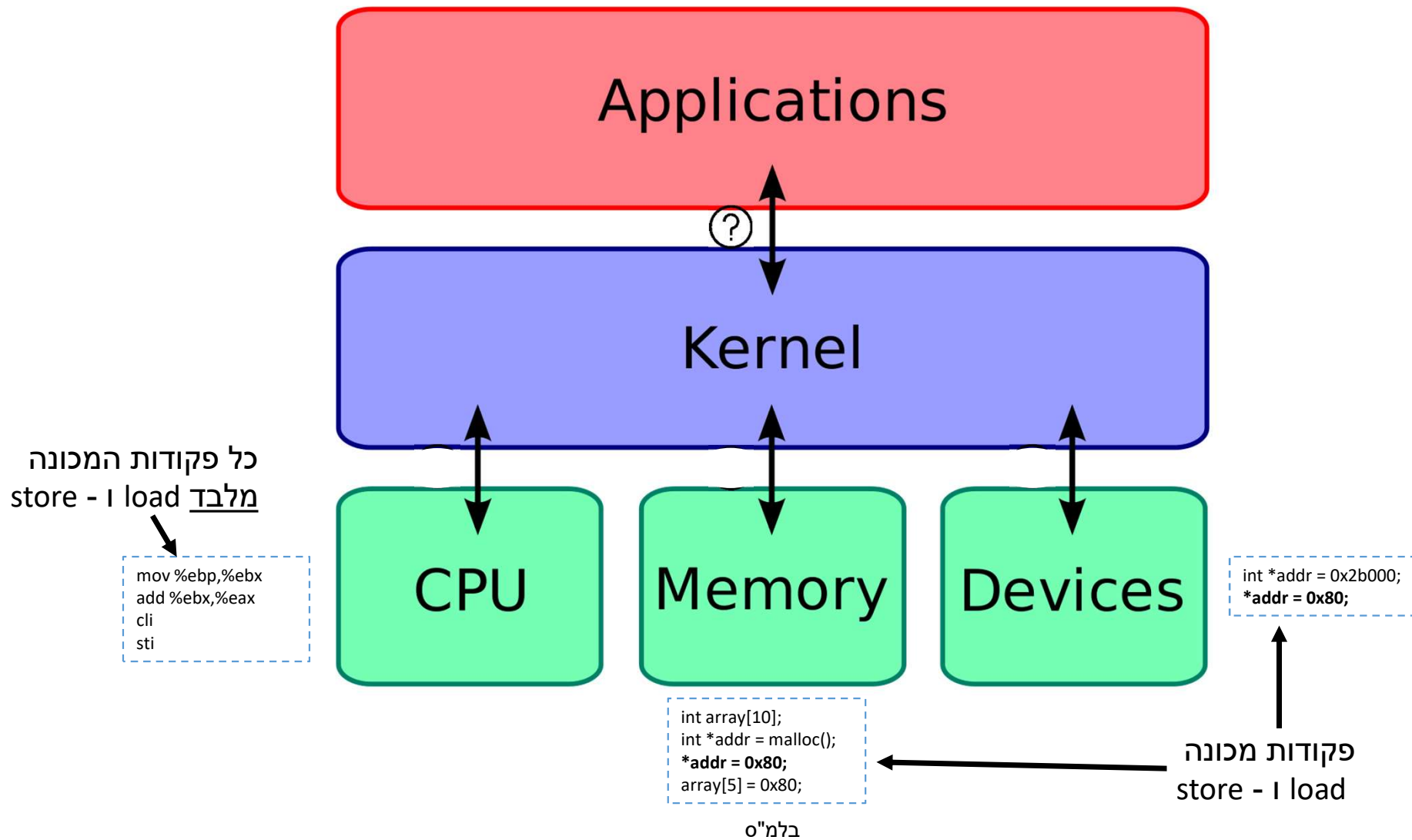
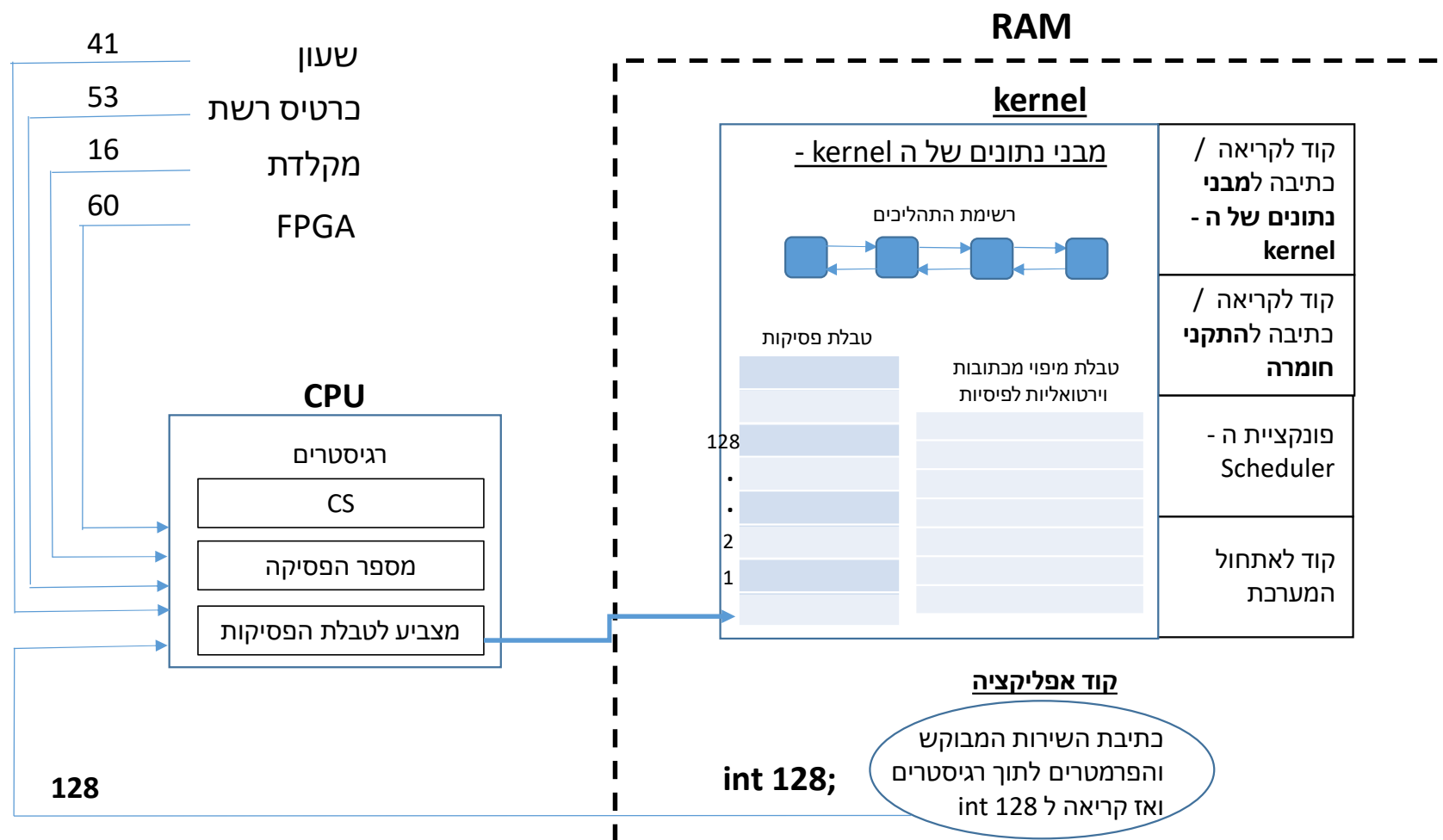


הרצאה 4 – קריאות מערכת



יחסי הגומלין בין ה kernel - לסביבתו



User space vs Kernel space

- קוד של מערכת ההפעלה רץ במוד חומרתי שנקרא Kernel space.
- קוד שאינו חלק מקוד ממערכת ההפעלה רץ במוד חומרתי שנקרא User space
 - כל הספריות שקיימות במערכת הקבצים
 - כלים שאנו משתמשים בהם
 - האפליקציות שאנו כותבים.
- איזה קוד אנו יכולים לכתוב שלא יגרור ריצת קוד של מערכת ההפעלה???
תשובה: ביצוע חישובים אריטמטיים וגישה למרחב הזיכרון של האפליקציה לצורך כתיבה/קריאה.

דוגמאות:

User and kernel code

```
printf("Hello World\n");
```

User code only

```
int *address = 0x2000;  
int value = *address;
```

User code only

```
int a, b, c;  
a = b + c;
```

User code only

```
char buf[20];  
sprintf(buf, "Hello World\n");
```

User and kernel code

```
sleep(1);
```

User code only

```
int buf[100];  
for(int i = 0; i < 100 ; i++)  
{  
    buf[i] = i;  
}
```

קריאת מערכת ב-Linux (1)

- המינוח המקובל – System Call (syscall בקיצור)
- תהליך משתמש מבצע קריאת מערכת על-מנת לבקש מגרעין מערכת ההפעלה לבצע עבורו שירות כלשהו
- מדוע שירות כלשהו חייב להתבצע דווקא על-ידי גרעין מערכת ההפעלה?
 - מצריך גישה לחומרה: קלט / פלט למסוף, קבצים, תקשורת ברשת, גרפיקה
 - מצריך גישה למבני נתונים של מערכת ההפעלה או של תהליכים אחרים: יצירת תהליכים חדשים, תקשורת בין תהליכים, גישה לנתוני משתמשים

קריאת מערכת ב-Linux (2)

- קריאת מערכת נראית כמו קריאה רגילה לפונקציה בקוד התוכנית
- לדוגמה: `open()` ו-`sprintf()` הן שתי פונקציות סטנדרטיות המהוות חלק מה-API (Application Programming Interface) של Linux וכלולות בספריה `libc`
- `sprintf()`, המשמשת להדפסת פלט לחוצץ, היא פונקציה רגילה, שאינה פונה לגרעין לצורך עבודתה
- אבל: `open()`, המשמשת לפתיחת קובץ, היא קריאת מערכת
- המתכנת, הכותב יישום שמשתמש בספריות מערכת ההפעלה, אינו מבדיל בין קריאת מערכת לבין קריאה רגילה לפונקציה. ההבדל נעוץ במימוש פונקציה לעומת מימוש קריאת מערכת

קריאת מערכת ב-Linux (3)

- כיצד ממומשת קריאת מערכת?
- התשובה: מעטפת קוד - code wrapper
- לדוגמה: הפונקציה `open()` היא מעטפת קוד, המפעילה באמצעות פסיקת תוכנה את קריאת המערכת שמבצעת את השירות המבוקש של פתיחת הקובץ, ומחזירה את תוצאת ביצוע השירות לקוד שקרא ל-`open()`
- פונקציות המעטפת הן אלו שמקושרות (linking) לתוכנית המשתמשת בהן

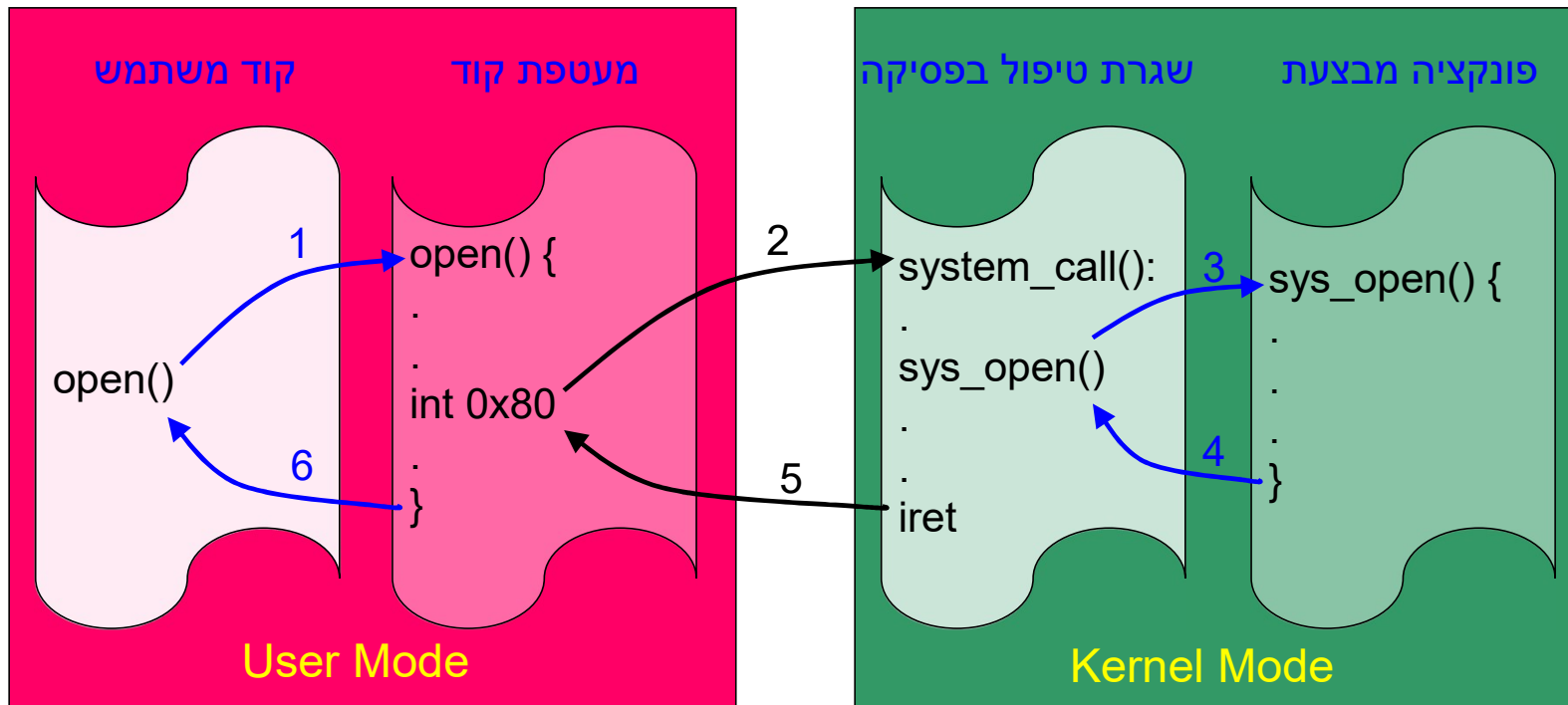
קריאת מערכת ב-Linux (4)

- הערך המוחזר על-ידי פונקצית המעטפת במקרה של הצלחת ביצוע השירות תלוי בסוג השירות המבוקש
- לדוגמה: `open()` מחזירה מספר מזהה (descriptor) לקובץ שנפתח. דרך מזהה זה ניתן כעת לקרוא ולכתוב לקובץ
- במקרה של כישלון, בדרך-כלל מוחזר הערך `-1`, וסוג השגיאה מוחזר במשתנה גלובלי הקרוי `errno`
- לדוגמה: אם הבקשה בקריאה ל-`open()` היתה לפתוח לקריאה קובץ שאינו קיים, יוכנס הערך `ENOENT` למשתנה `errno`, כאשר `ENOENT` הוא קבוע שערכו 2
- רשימת הקבועים המוחזרים במצבי כישלון עבור קריאת מערכת מסוימת זמינה דרך ה-`man page` של אותה קריאת מערכת
- רשימת ערכי הקבועים נמצאת בקובץ `/usr/include/errno.h` ובקבצים הכלולים בו

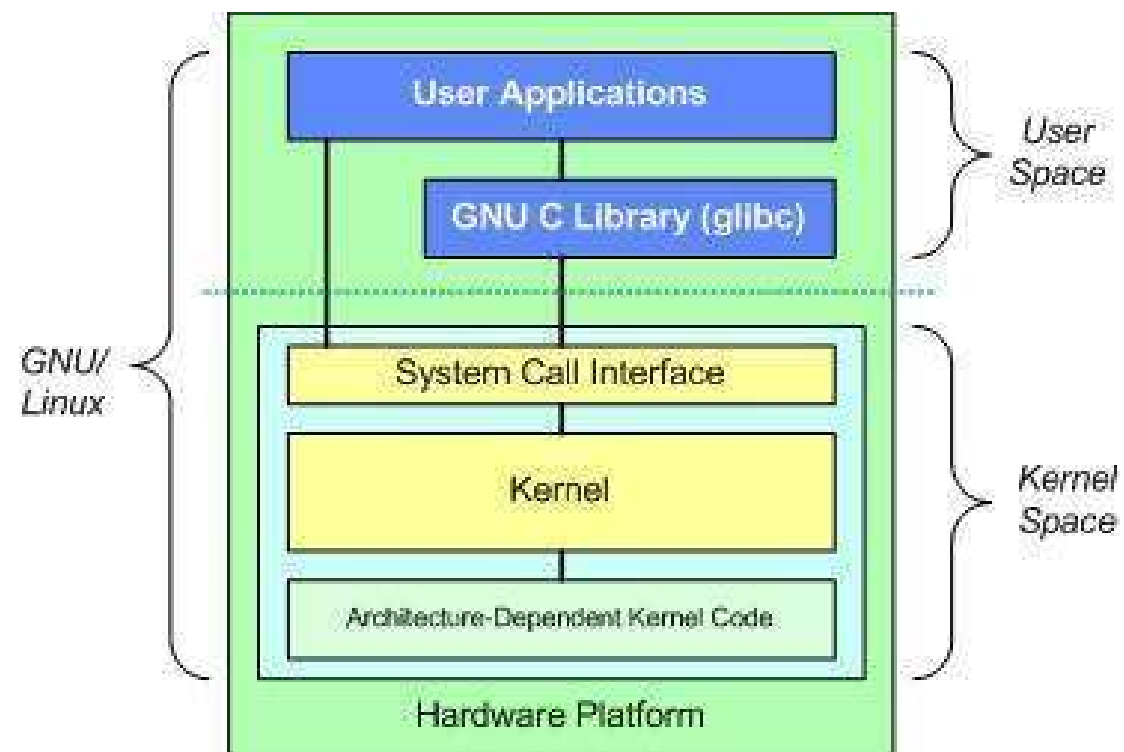
קריאת מערכת ב-Linux (5)

- כל קריאות מערכת ההפעלה ב-Linux מטופלות דרך פסיקת תוכנה אחת, מספר 128 (0x80)
- סוג השירות המבוקש נקבע באמצעות מספר שירות המועבר ברגיסטר `eax` על-ידי פונקצית המעטפת
- בתגובה לפסיקה המעבד עובר מ-`user mode` ל-`kernel mode` ומפעיל בגרעין את שגרת הטיפול בפסיקה הקרויה `system_call()` (קובץ גרעין: `arch/i386/kernel/entry.S`)
- השיגרה `system_call()` מנתבת את ביצוע הבקשה לפונקציה המתאימה לפי מספר השירות
- שם הפונקציה המבצעת את השירות מתאים לשם השירות עם הקידומת `"sys_"` לדוגמה `open()` מבוצעת על-ידי הפונקציה `sys_open()` קובץ גרעין: `fs/open.c`

קריאת מערכת ב-Linux (6)



פונקציות המעטפת והפונקציה בגרעין המבצעת את בקשת השירות הן פונקציות רגילות המופעלות לפי כללי קריאה לפונקציה שראינו בשקפים קודמים



בלמ"ס

